TransactiWar - Phase 2

Team ID: 07

Team members:

- Atish Kadam – CS25MTECH14003
- Akarsh Dubey – CS25MTECH14001
- Atharva Kale – CS25MTECH11024
- Prashant Kumar Dubey – CS25MTECH14011
- Debdip Choudhuri – CS25MTECH11025

Introduction

Application Overview

For this project, our team built **TransactiWar** — a web-based financial transaction platform that lets users send money to each other, manage their profiles, and keep track of their transaction history.
When a user signs up, they start with ₹100 in their account. From there, they can search for other users by username, transfer money to them with an optional comment, and view a full history of money sent and received. There's also a profile section where users can update their details and upload a profile picture.
The tech stack we used is straightforward vanilla PHP for the backend, MySQL as the database, and Apache as the web server, all running inside Docker containers so the setup is consistent across machines. The frontend uses Bootstrap 5 along with plain HTML, CSS, and JavaScript. One important constraint of this project was that we were not allowed to use any ready-made security libraries or frameworks — every security feature had to be written by us from scratch, which made the whole exercise significantly more challenging and educational.

Core Security Mechanisms

Since we had to implement security manually, we put considerable effort into covering the most common attack vectors from the start. Here's what we had in place before the War Game began:

1. Passwords are never stored in plain text. We used **bcrypt hashing** with a **cost factor of 12,** which is a well-established standard for securely storing credentials.

2. Input validation is enforced at every entry point. Usernames only accept letters, numbers, and underscores within a 3–30 character limit. Passwords must be at least 8 characters and include both letters and numbers. All inputs go through htmlspecialchars() and strip_tags() before being processed or displayed, which guards against XSS.

3. SQL injection is prevented entirely through PDO prepared statements with parameterized queries — no raw user input ever touches a query string.

4. CSRF protection is applied to every sensitive form. We generate a cryptographically random token per session using random_bytes(32) and validate it on every POST request using hash_equals(), which also prevents timing attacks during comparison.

5. Session management was something we paid particular attention to. Sessions are bound to the user's IP address and User-Agent at login time, so if either changes mid-session, the user is logged out immediately. We also regenerate the session ID on login to prevent session fixation, enforce a 30-minute inactivity timeout, and fully destroy the session on logout — both the server-side data and the cookie.

6. File uploads for profile pictures go through several layers of checks: file size is capped at 2MB, the extension is whitelisted, the real MIME type is verified from the file's magic bytes (not just the browser-reported type), and we scan the raw file contents for **embedded PHP or script tags.** On top of all this, the image is re-processed through PHP's GD library before saving, which strips any hidden metadata or payloads. Each uploaded file is also saved with a randomly generated name so it can't be directly guessed or targeted.

7. Access control is enforced on every protected page through a **require_login()** function that checks session validity before anything else is loaded.

8. Activity logging tracks every significant action logins, failed attempts, transfers, logouts recording the username, page, timestamp, and IP address to a log file. This turned out to be quite useful during the War Game for spotting suspicious activity.

War Game Attack Analysis

Attack by Other teams on our application

1. **Not able to upload high resolution image *(Invalid attack)* by Team 3**

   Our comment:-
   We have limited the pixel resolution of width*height<8000000 so you cannot add a high resolution image this is not a bug we are limiting it from our side if we dont add that people can do size attacks

so this is not a vulnerability, if you are able to exploit anything then that will become a vulnerability here that is not the case.

2. **Client-Controlled status Parameter and Array Injection Leading to Fake Profile Update Messages *(Invalid attack)* by Team 4**

3. **No Rate Limiting on Login Endpoint — Brute Force Attack Possible *(Invalid attack) by Team 18***

   Our comment:-
   Sir had mentioned in the class that password bruteforce attack will not be considered in this project that is why we havent implemented rate limiting and attackers are not allowed to perform DOS so we didnt find any need to implement this rate limiting

4. **Loss of User Access Caused by Non-Persistent Database Storage *(Invalid attack) by Team 14***

   Our Comment:-
   It might have happened that our site was down for a fix while he tried to login, I used his credentials and was able to login so there was no issue from our side.

5. **HTTPS Downgrade Attack *(Invalid attack) by Team 2***

   This attack is not valid as this attack was possible on all the 19 teams and it will happen cause we arent using domain name for our website.

6. **Web Server Version Disclosed in HTTP Response Header *(Invalid attack)* by Team 18**

   Our comment:-
   This nginx version is shown in the website itself when we first enable nginx or even if we get 502 error. So it is not a vulnerability.

Attacks Conducted By Us

1. **Atharva**

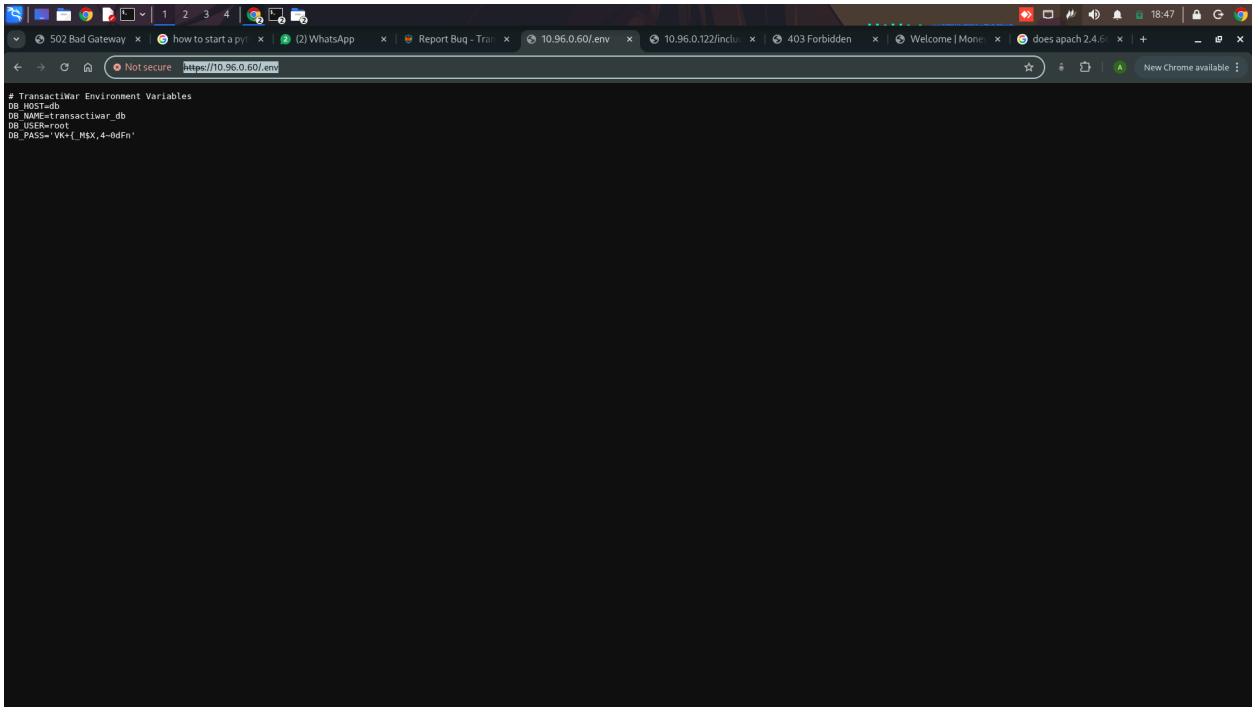
- **Vulnerability Exploited**

  ### “Directory traversal vulnerability”.
  Attack against team 11

  The application didnt prevent direct access of files present in the server.

- **Impact**

  In the url I appended /.env after the IP address and the entire content of .env files were visible on the web page which revealed sensitive information of their database.



- **Security Observations**

There was a server configuration issue in which the team did not restrict the users from accessing the server directories.

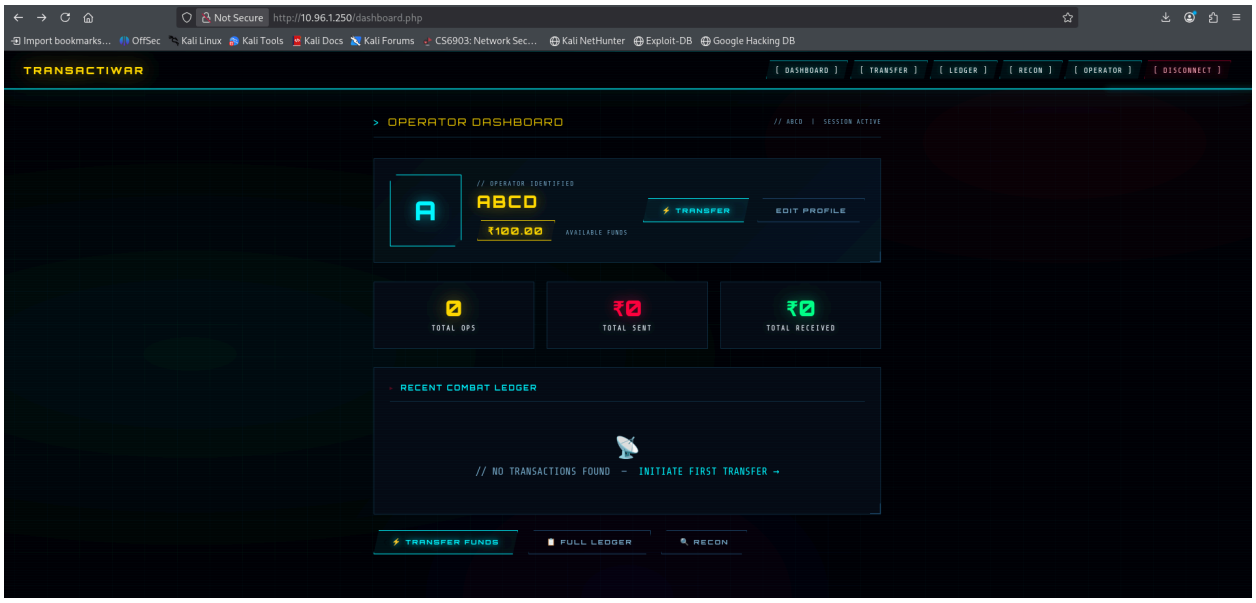
2. **Debdip**

“Application Served Over HTTP Allowing Man-in-the-Middle Attacks”on Against Team 13

The application is served entirely over HTTP instead of HTTPS, including sensitive areas such as the login page and authenticated user dashboard. This results in all data being transmitted in plaintext over the network.

An attacker positioned on the same network (e.g., public Wi-Fi) can intercept or modify traffic via a Man-in-the-Middle (MITM) attack. This could lead to credential theft, session hijacking, and unauthorized account access. Additionally, the lack of transport encryption allows attackers to inject malicious content into responses.

This issue represents a failure to enforce secure transport (HTTPS) and exposes users to significant confidentiality and integrity risks. It is recommended to enforce HTTPS across the entire application, implement automatic HTTP-to-HTTPS redirection, and ensure secure cookie attributes are properly configured.



3. Akarsh

Vulnerability Exploited

“Downgrade attack of cipher suite “ on Against Team 16

Steps to reproduce:

Open a terminal with OpenSSL installed.

Execute the following command to simulate an attacker explicitly requesting a weak cipher suite over TLSv1.2

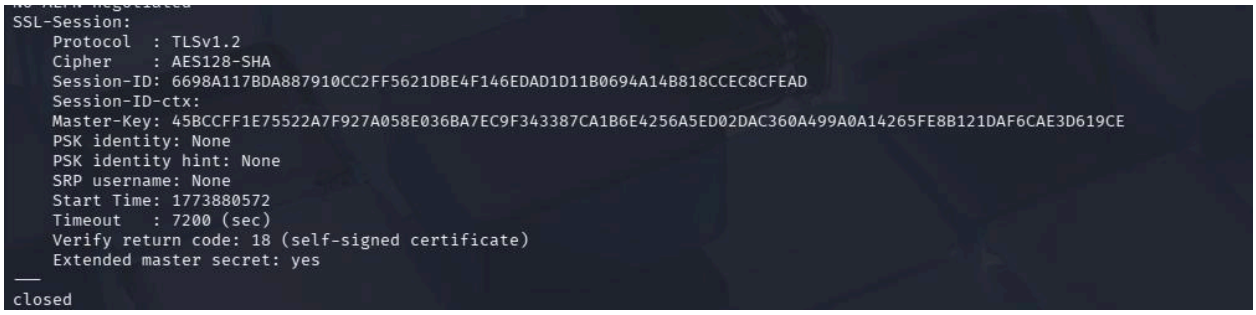
openssl s_client -connect 10.96.1.96:443 -tls1_2 -cipher 'AES128-SHA'

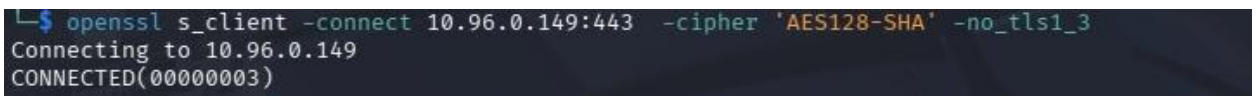
Observe the SSL-Session block in the resulting output to verify which protocol and cipher the server agreed to use.

Expected Behaviour:
The server should reject the connection (handshake failure).
Actual Behaviour: The server accepts the client's preference and successfully completes the handshake using the outdated AES128-SHA cipher (TLS_RSA_WITH_AES_128_CBC_SHA).

Impact/Severity:
Because the server respects the client's cipher preference, an active Man-in-the-Middle (MitM) attacker can modify a victim's initial ClientHello packet to strip away strong ciphers. This forces a downgrade to weak cryptography. Using AES128-SHA strips away Perfect Forward Secrecy (by using RSA key exchange instead of ECDHE), meaning if the server's private key is ever compromised, all past traffic can be decrypted. Furthermore, it forces the use of CBC mode, which is historically vulnerable to padding oracle attacks (such as Lucky13) in TLSv1.2.

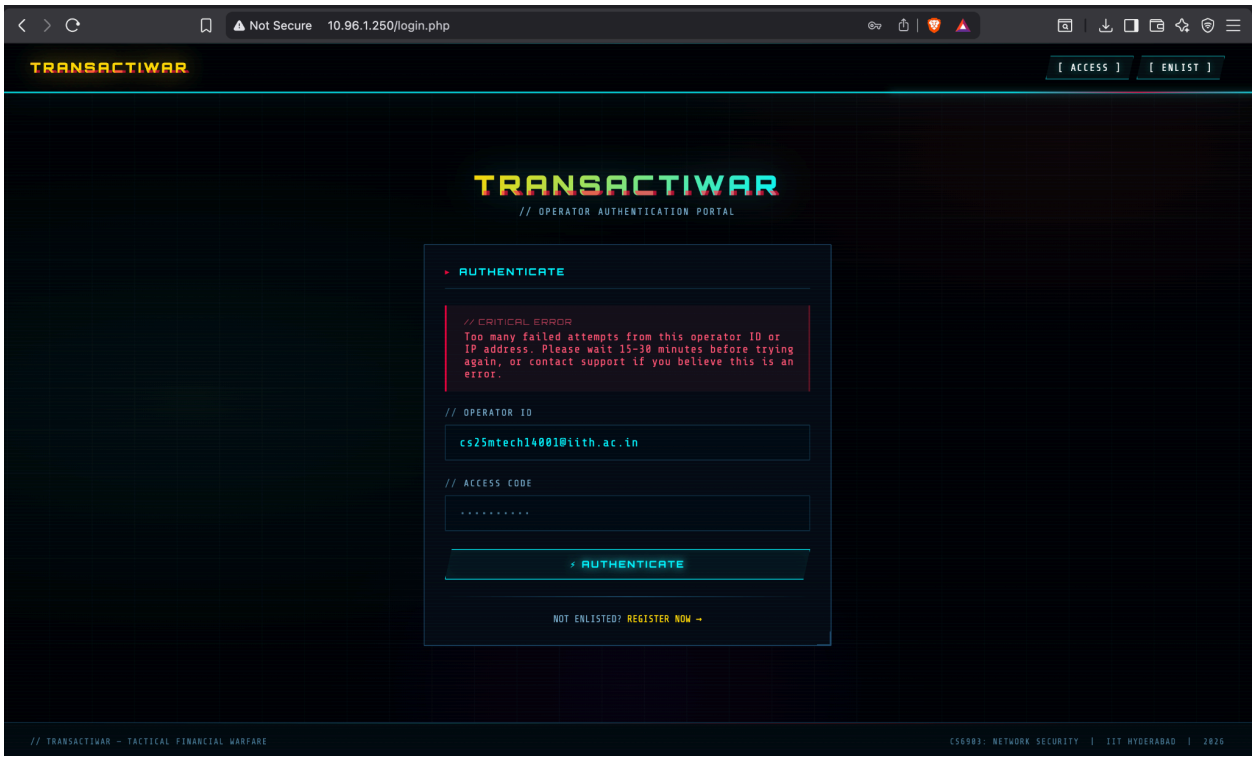
Output of OpenSSL command success to force cipher suite downgrade is attached.(The second screenshot shows that server accepted AES128-SHA cipher)

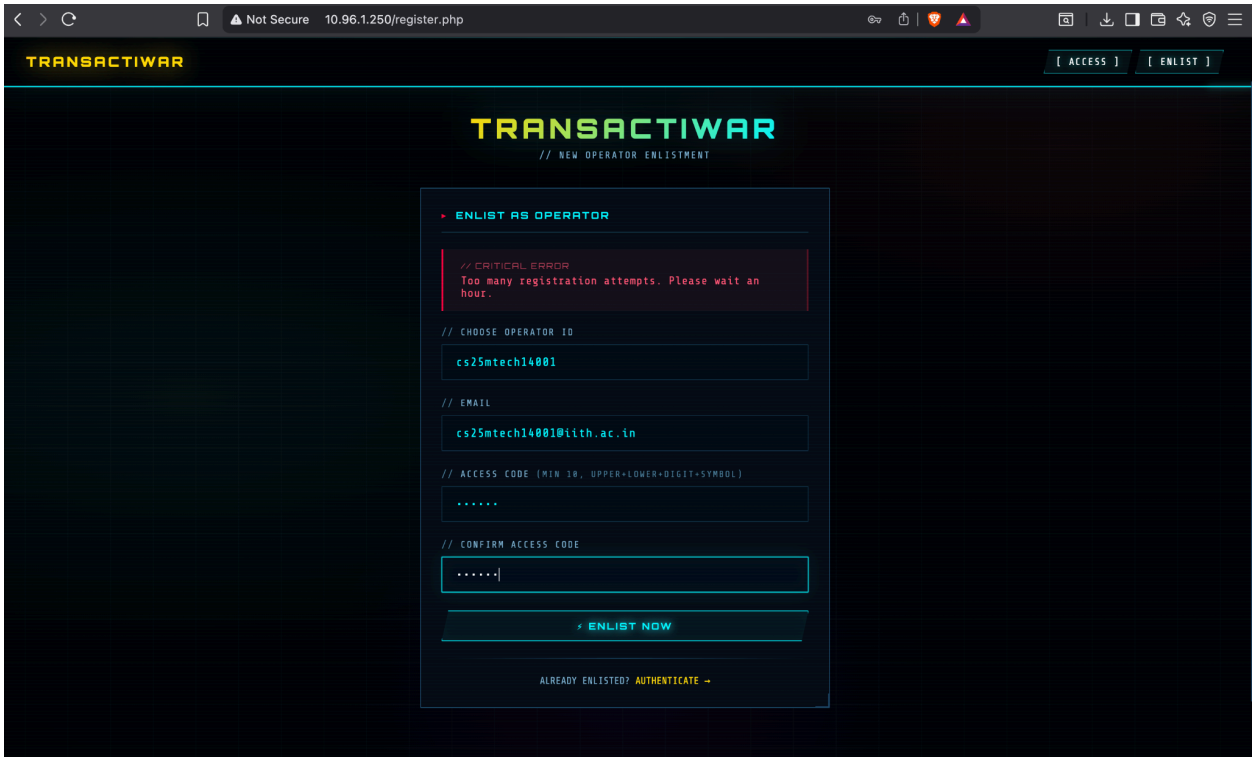




-> “Not allowed to login by the credential provided by sir as well as unable to register also” on Against Team 13

.when i login the first time this shows you have attempted many times as well when i am registering it shows "Too many registration attempts. Please wait an hour."

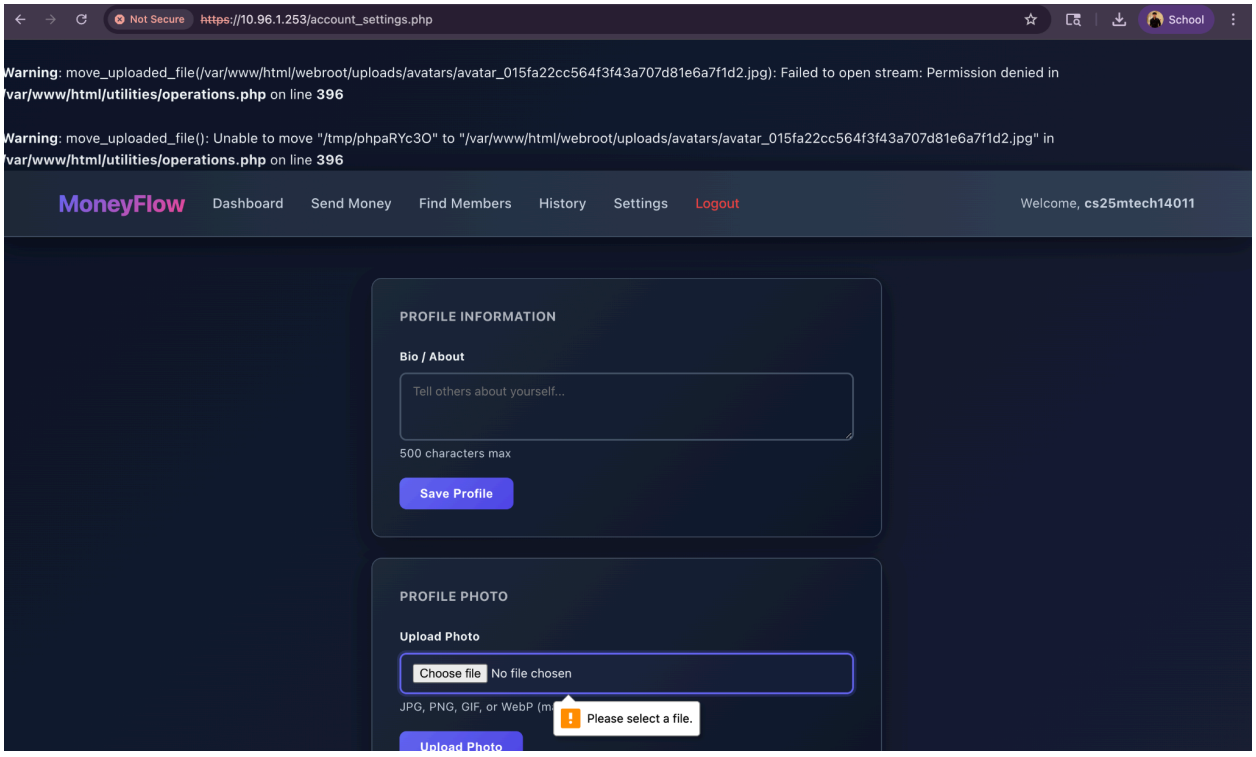



Not Secure
10.96.1.250/register.php
TRANSACTIWAR
[ACCESS]
[ENLIST]
TRANSACTIWAR
// NEW OPERATOR ENLISTMENT
ENLIST AS OPERATOR
// CRITICAL ERROR
Too many registration attempts. Please wait an hour.
// CHOOSE OPERATOR ID
cs25mtech14001
// EMAIL
cs25mtech14001@iith.ac.in
// ACCESS CODE (MIN 10, UPPER+LOWER+DIGIT+SYMBOL)
// CONFIRM ACCESS CODE
ENLIST NOW
ALREADY ENLISTED? AUTHENTICATE →

4. Prashant

Improper File Upload Handling Due to Insufficient File Permissions : Against Team-9

The application fails to process file uploads due to insufficient write permissions on the target directory (/var/www/html/webroot/uploads/avatars/). This results in a “Permission denied” error when attempting to move the uploaded file using move_uploaded_file(). Additionally, verbose warning messages expose internal file paths and backend script locations (/utilities/operations.php), leading to an information disclosure vulnerability. This indicates both improper server-side permission configuration and insecure error handling in the file upload functionality.



5. **Atish**

“Verbose Error Message Leading to Sensitive Information Disclosure “ founded by me and attacked by my teammate Prashant. Against Team 1

The error indicates a database connection failure (SQLSTATE[HY000] [2002] Connection refused), likely due to the database server being unreachable or misconfigured. More importantly, it exposes sensitive internal details such as file paths (/var/www/html/config/db.php), backend function calls, and application structure. This results in an information disclosure vulnerability, which can help attackers understand the system architecture and craft targeted attacks.

Security Patching & Enhancements

There were no security vulnerabilities found in our application. But a security enhancement is the use of a Trusted Certificate Authority (CA) for SSL/TLS certificates in a production environment. Instead of relying on self-signed certificates, using a Trusted CA ensures that the certificates are recognized by browsers and operating systems, preventing security warnings and trust issues.

Additionally, Trusted CAs provide protection against Man-in-the-Middle (MITM) attacks by verifying the authenticity of the website, ensuring that encrypted communication remains secure. Implementing this measure will enhance the credibility and security of the TransactiWar platform, ensuring a safer and more reliable user experience.

Lessons Learned & Future Security Enhancements

Use Trusted CA in Production Environment
In a production environment, it is essential to use a Trusted Certificate Authority (CA) for SSL/TLS certificates instead of self-signed certificates. Trusted CAs ensure that certificates are recognized by browsers and operating systems, preventing security warnings and trust issues. They also protect against Man-in-the-Middle (MITM) attacks by verifying the authenticity of the website, ensuring encrypted communication remains secure.

A proper domain name would also be registered for the application, as HTTPS and CA-signed certificates require a domain to function correctly. Without a domain name, it is not possible to obtain a certificate from a Trusted CA or configure HTTPS properly — which was also the reason the HTTPS downgrade attack submitted against our application was considered invalid across all 19 teams.
Once a domain is in place, we would submit it to the HSTS Preload List (hstspreload.org). HSTS (HTTP Strict Transport Security) forces browsers to always connect to the site over HTTPS, even on the very first visit. Being on the preload list takes this a step further — the domain is hardcoded into browsers like Chrome and Firefox, meaning there is no initial HTTP request at all, completely eliminating any window for a downgrade attack or SSL stripping.

Contributions

Name	Roll No	Contribution
Atish Kadam	CS25MTECH14003	Implemented User Authentication (Login and Register) and Session Management
Akarsh Dubey	CS25MTECH14001	Designed Database, Configured Docker, Implemented Dashboard, and Added Logging Feature
Atharva Kale	CS25MTECH11024	Implemented Transactions and Removed Directory Traversal Vulnerability
Prashant Kumar Dubey	CS25MTECH14011	Implemented Transaction History
Debdip Choudhuri	CS25MTECH11025	Implemented Profile Management

Github link - https://github.com/cs25mtech14003-ux/Team-7

References

- PHP Manual: https://www.php.net/manual/
- Docker Documentation: https://docs.docker.com/
- Let's Encrypt Documentation: https://letsencrypt.org/docs/
- HSTS Preload List: https://hstspreload.org/
- LLMs

ANTI-PLAGIARISM STATEMENT

We certify that this assignment/report is our own work, based on our personal study and/or research, and that we have acknowledged all material and sources used in its preparation, whether books, articles, packages, datasets, reports, lecture notes, or any other document, electronic or personal communication. We also certify that this assignment/report has not previously been submitted for assessment/project in any other course lab, except where specific permission has been granted from all course instructors involved, or at any other time in this course, and that we have not copied in part or whole or otherwise plagiarized the work of other students and/or persons. We pledge to uphold the principles of honesty and responsibility at CSE@IITH. In addition, we understand our responsibility to report honor violations by other students if we become aware of them.

Names:

- Atish Kadam – CS25MTECH14003
- Akarsh Dubey – CS25MTECH14001
- Atharva Kale – CS25MTECH11024
- Prashant Kumar Dubey – CS25MTECH14011
- Debdip Choudhuri – CS25MTECH11025

Date: 26-03-2026
Signature: Atish, Akarsh,Atharva, Prashant , Debdip